

SOLUCIONES INFORMÁTICAS PARA LA EMPRESA

TAREA 2

Integración, automatización de procesos e IA agéntica aplicada al back-office

UCAM Stereo

Integración empresarial y automatización del e-commerce académico

AUTORES

Dusjan Jevtic Santander
Germán Navarro Marín
Javier Mínguez García
Pedro Javier Moreno Gázquez
Víctor Matías García

PROFESOR

Antonio Serrano Fernández

TITULACIÓN

Grado en Ingeniería Informática
Modalidad presencial

Curso 2026/2027

Aplicación: `www.ucamstereo.onl` **Repositorio:** `github.com/germannvrrocalvo/PaginaVenta2.0`

Prototipo académico sin actividad comercial real. Los datos y pagos utilizados pertenecen a un entorno de prueba (PayPal Sandbox).

Universidad Católica San Antonio de Murcia (UCAM) · Noviembre de 2026

1 Resumen, objetivo y punto de partida

Nuestro objetivo. Ampliamos la tienda UCAM Stereo de la Tarea 1 con un back-office independiente, una integración estructurada basada en eventos, automatizaciones en n8n, la gestión supervisada de incidencias mediante un agente de IA y un cuadro de mando operativo calculado sobre los registros del propio sistema.

1.1 Solución heredada de la Tarea 1

En la Tarea 1 desarrollamos un e-commerce académico de audio (vinilos, tocadiscos, altavoces y auriculares) con interfaz React/Vite, API Node.js/Express y PostgreSQL. Implementamos autenticación, catálogo y variantes, carrito, checkout, PayPal Sandbox, pedidos, soporte y eventos instrumentados. Publicamos la tienda en DonDominio, desplegamos la API en Render y usamos un n8n local, expuesto mediante ngrok, para enviar el correo de confirmación. El enunciado de la Tarea 2 exige que la automatización y el back-office estén desplegados fuera de nuestros equipos, por lo que sustituimos ese n8n local por n8n Cloud.

Elemento	Herencia y nueva aportación
Front-office	Reutilizamos la tienda, el modelo transaccional y los eventos de pedido y soporte.
Motor de eventos	Incorporamos una *outbox* transaccional (`integration_outbox`) que entrega webhooks a n8n sin bloquear la compra.
Back-office	Base PostgreSQL independiente en Neon y panel CoreAdmin con acceso propio.
Automatización	Flujos de n8n Cloud que validan, transforman, registran y notifican.
Observabilidad	Chartbrew, incrustado en CoreAdmin, consulta vistas SQL de solo lectura; conservamos como antecedente un export de Grafana.
IA operativa	Análisis de incidencias con un agente de n8n, propuesta registrada, revisión humana en CoreAdmin y ejecución posterior separada.

Accesos documentados: tienda www.ucamstereo.onl · repositorio Tarea 2 GitHub/PaginaVenta2.0. Las URL públicas de n8n, CoreAdmin y del panel de métricas se indican en la entrega del Campus Virtual.

1.2 Declaración del punto de partida

Partimos del repositorio PaginaVenta (Tarea 1) y de la aplicación UCAM Stereo que desarrollamos en la Tarea 1. Para la Tarea 2 creamos el repositorio PaginaVenta2.0 el 31/10/2026 e incorporamos el código de la Tarea 1 el 01/11/2026, junto con la primera automatización de pedidos con n8n. Como declaramos en la memoria anterior, **no utilizamos un e-commerce comercial completo como punto de partida**: creamos el frontend, la API, las reglas de negocio y el modelo de datos, tomando como referencia visual un diseño de Figma Make. Sobre esa base desarrollamos en esta tarea el back-office, CoreAdmin, los flujos de n8n y el cuadro de mando. Utilizamos asistentes de IA generativa como apoyo durante el diseño, la implementación, la depuración y la redacción, y revisamos los cambios antes de incorporarlos; el detalle figura en el Anexo A.

2 Arquitectura e intercambio de datos

2.1 Separación entre front-office y back-office

Mantenemos **dos bases PostgreSQL independientes**: la de la tienda, con sus pedidos y la `integration_outbox`, y la del back-office en Neon. El backend Express de Render conecta con ambas mediante variables distintas (`DATABASE_URL` y `BACKOFFICE_DATABASE_URL`). Los eventos de la tienda llegan a n8n mediante webhooks autenticados; n8n valida los datos y actualiza el back-office. CoreAdmin, nuestro panel de gestión, inicia sesión con cuentas propias del back-office y opera mediante endpoints autenticados del mismo backend. Las métricas se muestran en **Chartbrew**, incrustado en CoreAdmin.

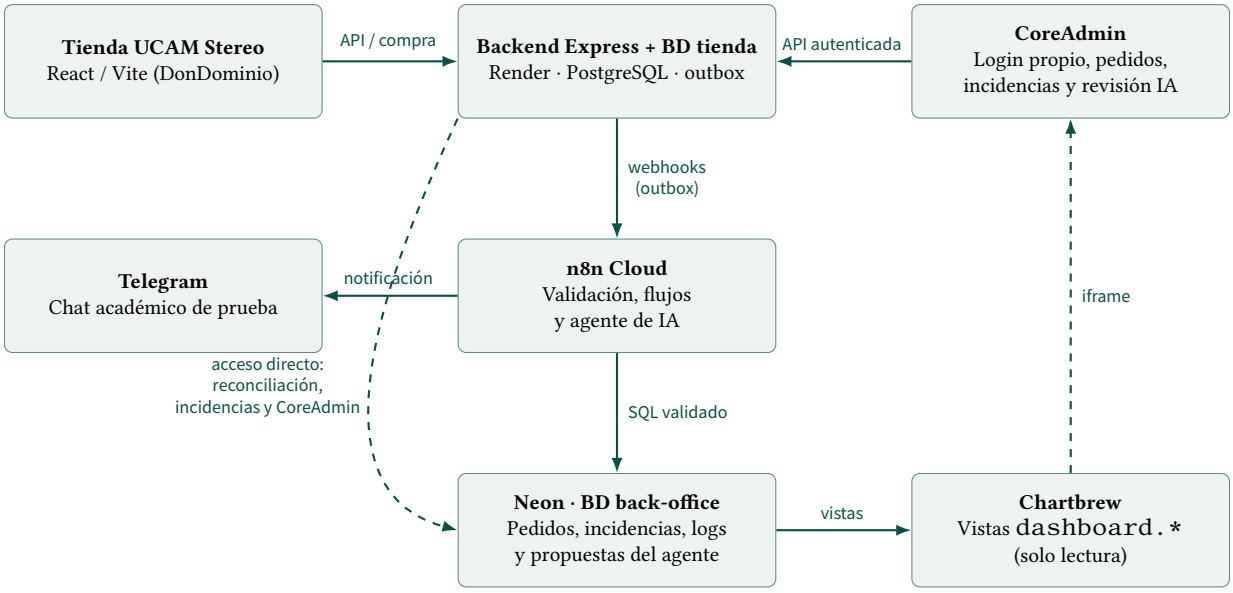


Figura 1. Flujo de datos. Las flechas continuas son los intercambios por eventos y consultas de lectura; la discontinua desde el backend representa el acceso directo a la BD del back-office que describimos en 2.2.

2.2 Persistencia y acceso

La base del back-office contiene `orders`, `order_items`, `integration_logs`, `incidents`, `agent_actions`, `agent_action_outbox`, `backoffice_admins` y `backoffice_sessions`. El archivo `dashboard.sql` crea el esquema `dashboard` con diez vistas para indicadores, y el usuario `dashboard_reader` solo puede leer esas vistas.

Además del recorrido por n8n, el backend escribe directamente en el back-office en dos casos: (1) `backoffice-order-reconciliation.js` copia cada 30 segundos todos los pedidos de la tienda, con sus líneas y su pago, a `orders` y `order_items`; y (2) `backoffice-incident-sync.js` replica los tickets de soporte con reintentos. Diseñamos esta reconciliación como red de seguridad frente a eventos perdidos, pero tiene consecuencias que asumimos: **un pedido o una incidencia presente en Neon no demuestra por sí solo que haya pasado por n8n**; la prueba de cada escenario debe enlazar el `event_id` con la ejecución de n8n y su fila en `integration_logs`.

3 Contratos de integración y validaciones

3.1 Contrato del evento `order.created`

Documentamos en `docs/N8N_PEDIDOS.md` el contrato que genera `buildOrderCreatedEvent`: identificador de evento determinista, tipo, marca temporal y datos del pedido. Lo enviamos por `POST` al webhook con la cabecera `Authorization: Bearer` y la cabecera `Idempotency-Key`, igual al `event_id`. Mostramos el ejemplo de esa guía:

```
{
  "event_id": "order.created:UC-2026-01004",
  "event_type": "order.created",
  "timestamp": "2026-11-01T10:00:00.000Z",
  "order": {
    "id": "UC-2026-01004", "customer_id": "42",
    "customer_email": "cliente@example.com",
    "items": [{ "product_id": "PROD-002",
      "name": "Auriculares de prueba", "quantity": 1,
      "unit_price": 79.99 }],
    "total": 79.99, "currency": "EUR",
    "status": "pending",
    "created_at": "2026-11-01T10:00:00.000Z"
  }
}
```

3.2 Catálogo de eventos implementados

Evento	Emisor y momento	Contenido y destino
`order.created`	Transacción que crea el pedido en la tienda (estado `creado`, antes de capturar el pago).	Contrato anterior; webhook de pedidos de n8n.
`order.status_changed`	Misma transacción que el cambio de estado: captura PayPal (`creado`→`pagado`), cambio administrativo o envío automático cada 3 h (`pagado`→`enviado`).	`event_id` `order.status_changed`:*pedido*:*uuid*; `previous_status`, `status`, `changed_at`, `change_source`. Flujo 15.
`support.requested`	Transacción que guarda el ticket de soporte.	`incident`: UUID, referencia, pedido indicado, nombre, correo, asunto, descripción, prioridad, estado y chat de Telegram. Flujo 16.
`agent.action.approved`	Transacción de aprobación humana en CoreAdmin (outbox del back-office).	Incidencia y acción aprobada con revisor y fecha. Flujo «Telegram tras aprobación».

Además, el análisis del agente es una petición síncrona que solo envía un JSON con el campo `incident_id` (UUID) con una cabecera secreta configurable (sección 5).

3.3 Validaciones y respuestas

En el backend. Antes de enviar exigimos URL HTTPS en producción y secreto configurado; no seguimos redirecciones y cortamos la petición a los 5 s. Cualquier respuesta distinta de 2xx deja el evento en `retry`.

En n8n (exports `docs/n8n/15-*` y `16-*`). Cada flujo comprueba que el cuerpo sea un objeto JSON, la presencia de los campos obligatorios, el `event_type` esperado, las fechas ISO y, en incidencias, el formato UUID y el correo. Las respuestas siguen un mismo patrón:

- **400** `rejected`: payload inválido, con `missing_fields` e `invalid_fields`.
- **200** `completed` o `already_processed`: operación aplicada o duplicado detectado por `event_id`.
- **503** `pending_sync` / `retry`: el pedido aún no existe en el back-office o Telegram ha fallado; el backend lo reintenta.
- **409**: el flujo de Telegram vuelve a consultar Neon y se niega a enviar si la acción no figura como `approved`.

4 Escenarios de automatización obligatorios

4.1 Escenario 1 · Pedido generado e integrado

Disparador: al iniciar el pago con PayPal Sandbox, la tienda crea el pedido y, en la misma transacción, encola `order.created`. Tras el `COMMIT`, la outbox entrega el evento al webhook de pedidos; según el README y `docs/N8N_PEDIDOS.md`, ese flujo valida, normaliza, comprueba duplicados y registra pedido, líneas y log. La confirmación del pago llega después como `order.status_changed` (escenario 2). El grupo comunicó en su checklist del 04/11 una compra real integrada; como la reconciliación también copia pedidos, la evidencia debe mostrar la ejecución de n8n y su registro, no solo la fila en `orders`.

4.2 Escenario 2 · Cambio operativo y sincronización

El backend emite `order.status_changed` cuando cambia el estado de un pedido. Además, para simular la expedición del almacén, una automatización del backend pasa cada tres horas un pedido pagado, elegido al azar, a `enviado` y emite el evento con `change_source = automatic_shipping_bot`; así el escenario se produce sin intervención manual. El flujo «SIE - Cambio automático de estado de pedido» valida el evento y, en una única sentencia SQL, actualiza `orders.order_status` y escribe el log. Si el evento ya tiene un registro correcto, responde `already_processed` sin repetir la operación. Si el pedido todavía no existe en el back-office, registra `pending_sync` con el error `order_not_found` y responde 503, de modo que la outbox lo reintenta: es el «evento pendiente de sincronización» que contempla el enunciado. Según `docs/ESCENARIOS_15_16.md`, el flujo está publicado.

4.3 Escenario 3 · Incidencia, postventa y Telegram

El formulario de soporte guarda el ticket y encola `support.requested`. El flujo «SIE - Incidencia y Telegram»:

1. Valida el evento y crea o actualiza la incidencia en `incidents`, asociada al pedido indicado y al correo del cliente.
2. Envía un aviso al chat académico de Telegram configurado en `TELEGRAM_TEST_CHAT_ID`.
3. Registra `telegram_message_id` y el log `completed`. Si Telegram falla, registra `failed`/`telegram_error` y responde 503 para reintentar.

No reenvía avisos si la incidencia ya tiene mensaje. El seguimiento continúa en CoreAdmin con el agente y la revisión humana, y un segundo flujo comunica por Telegram la propuesta aprobada.

5 IA agéntica y supervisión humana

5.1 Papel operativo del agente

Integramos el agente en el seguimiento de incidencias. En CoreAdmin, un administrador pulsa *Analizar con IA*. El backend comprueba su sesión, marca la incidencia como `running` (lo que impide análisis simultáneos durante cinco minutos) y llama al webhook del agente enviando solo el UUID de la incidencia, con un tiempo máximo de 60 s. Según `docs/N8N_AGENTE_INCIDENCIAS.md`, el flujo de n8n valida el UUID, consulta la incidencia en el back-office (`consultar_incidencia`) y registra su propuesta en `agent_actions` (`registrar_propuesta`) con `review_status = pending_review`. Si n8n termina sin registrar propuesta, el backend marca el análisis como fallido y muestra el error.

Requisito del enunciado	Implementación
Herramienta de consulta	`consultar_incidencia` sobre la BD del back-office (credencial distinta de la de la tienda).
Herramienta de propuesta	`registrar_propuesta`, que inserta la propuesta en `agent_actions`. El agente no dispone de herramienta de envío.
Salida estructurada	Columnas `case_type` (caso), `consulted_data` (datos consultados), `proposed_action`, `justification`, `confidence`, `risks` y estado final.
Estado final	`review_status`: `pending_review`, `approved` o `rejected`; `execution_status`: `not_requested`, `pending`, `executed`, `retry`, `dead_letter` o `not_applicable`.
Registro y auditoría	`reviewed_by`, `reviewed_at`, `executed_at`, `execution_error` y `telegram_message_id`.

5.2 Límites de actuación y supervisión humana

Separamos proponer, aprobar y ejecutar:

- **Aprobación en una única transacción.** Una aprobación en CoreAdmin bloquea la propuesta pendiente (`FOR UPDATE`), guarda el revisor y la fecha, pone `execution_status = pending` y encola `agent.action.approved` en `agent_action_outbox` dentro de esa transacción. Un rechazo deja la acción en `not_applicable` y no encola nada.
- **Comprobación previa al envío.** El flujo de Telegram vuelve a leer la acción en Neon y solo envía si sigue `approved`. Si el envío falla, la decisión humana se conserva y el backend reintenta hasta ocho veces.
- **Nada autónomo sobre pagos.** No autorizamos cambios autónomos de pagos, importes ni estados de pedido.

Tomamos como referencias el marco de gestión de riesgos de IA del NIST (*AI RMF 1.0*) y, de forma orientativa, el art. 14 del Reglamento (UE) 2024/1689; *no afirmamos que nuestro prototipo sea un sistema de alto riesgo regulado.*

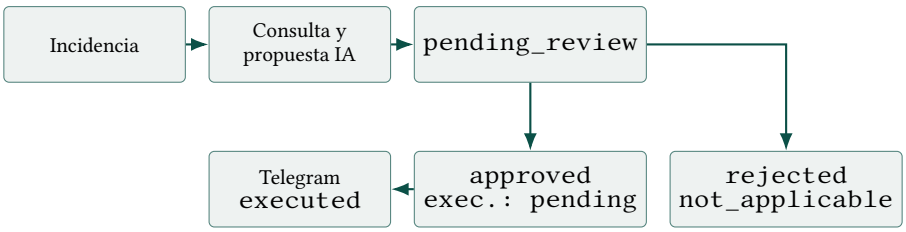


Figura 2. Ciclo de una propuesta: la revisión humana (`review_status`) y la ejecución (`execution_status`) se registran por separado.

6 Cuadro de mando y métricas operativas

Incorporamos **Chartbrew** en la página Dashboard de CoreAdmin mediante una vista incrustada. Chartbrew se conecta a Neon con `dashboard_reader` y cada gráfico lee una de las diez vistas de `dashboard.sql`; no introducimos cifras a mano. Seleccionamos las siguientes métricas:

Métrica	Vista / cálculo	Interpretación
Pedidos totales, importe y ticket medio	`kpis`: `count`, `sum` y `avg` de `total_amount` (pagados)	Volumen y valor de las ventas simuladas; el valor medio del pedido es un indicador básico del comercio electrónico (Shopify).
Pedidos por día (gráfico)	`pedidos_por_dia`, últimos 30 días	Evolución de la actividad; gráfico exigido por el enunciado.
Productos más vendidos	`productos_mas_vendidos`	Unidades e importe por producto en pedidos pagados.
Éxito de integración y errores por tipo	`eventos_integracion`, `errores_por_tipo`	Tráfico y errores, dos de las «señales doradas» de *Site Reliability Engineering* (Google); los duplicados descartados cuentan como bien gestionados, según el principio de idempotencia (Stripe).
Incidencias abiertas y tiempo medio de resolución	`kpis` sobre `resolved_at`	Carga de soporte y tiempo medio de resolución (MTTR), métricas clásicas de gestión de incidencias (Atlassian).
Aprobación del agente	`agente_por_estado`	Propuestas aprobadas sobre el total: supervisión humana de la IA (NIST AI RMF; Reglamento (UE) 2024/1689, art. 14).

6.1 Interpretación y límites de los datos

- **Pedidos.** Las vistas filtran `source_system = 'store'`, valor que asigna la reconciliación a todas las filas. Por tanto miden la actividad comercial completa, *no* los pedidos que llegaron a través de n8n; la integración se mide con `integration_logs`.
- **Tiempo de integración.** La vista `kpis` incluye un tiempo medio de integración (`integrated_at - created_at`). Lo excluimos porque la reconciliación reescribe `integrated_at` cada 30 s. Lo comprobamos en una base local con los datos de prueba: el valor crece en cada ciclo y no mide la integración.
- **Eventos.** Solo `completed` cuenta como correcto y `already_processed` como duplicado; cualquier otro valor, incluido `pending_sync`, cuenta como fallido. Los rechazos 400 de validación no generan fila en `integration_logs`, de modo que no aparecen en esta métrica (sección 8).
- **Incidencias.** Una incidencia está abierta mientras `resolved_at` es nulo. CoreAdmin rellena esa fecha al pasarla a resuelta o cerrada.

7 Matriz de decisión de implantación

Elegimos la arquitectura híbrida (B), que obtiene **3,84** puntos sobre 5 frente a los **3,29** de la API propia con base de datos independiente (A). Como ambas usan n8n, empatan en integración y trazabilidad; la ventaja de B está en la adecuación al tamaño de la empresa, la funcionalidad y el menor esfuerzo de implantación. A cambio, B tiene menos margen para escalar a medida.

7.1 Alternativas comparadas

Comparamos dos enfoques que llegamos a trabajar durante el desarrollo, de modo que la valoración parte de lo que vimos al construirlos:

- **A · API propia con base de datos independiente (desarrollo propio).** Creamos para CoreAdmin una API con su propia base PostgreSQL. n8n llamaba a sus endpoints para registrar pedidos, cambios de estado e incidencias, y la API validaba y normalizaba los datos, descartaba eventos repetidos y anotaba cada llamada en su propia tabla de registro (`registro_integracion`). La probamos y la desplegamos, pero la retiramos el 3 de noviembre de 2026 para no tener dos back-offices. No debe confundirse con las rutas de back-office que CoreAdmin usa ahora en el backend compartido.
- **B · Arquitectura híbrida con aplicación propia y automatización externa.** Conservamos la tienda y su backend. Los eventos se guardan en una outbox y se envían a n8n, que los valida y los registra en la base independiente `sie-backoffice-db`, en Neon. El backend replica además los tickets de soporte y reconcilia periódicamente los pedidos en esa base. CoreAdmin gestiona las incidencias y la revisión humana a través del backend compartido, y Chartbrew muestra las métricas.

No incluimos soluciones SaaS ni ERP/CRM porque no las hemos probado y sus notas no tendrían el mismo respaldo.

7.2 Cómo hemos puntuado

Cada alternativa recibe una nota de 1 (muy desfavorable) a 5 (muy favorable) en los criterios que propone el enunciado. La nota ponderada es el peso multiplicado por la nota, y el total es la suma de esas cifras. Las notas son una valoración razonada del grupo, no mediciones, y juzgan el enfoque, no lo terminada que esté cada pieza. Para valorar la adecuación tratamos UCAM Stereo como una tienda pequeña, ya que el caso es académico y no fija un tamaño.

Criterio	Peso	Nota A	Nota B	Ponderada A	Ponderada B
Funcionalidad	12%	4	5	0,48	0,60
Coste estimado	8%	3	3	0,24	0,24
Esfuerzo de implantación	10%	2	3	0,20	0,30
Escalabilidad	4%	4	3	0,16	0,12
Integración	15%	4	4	0,60	0,60
Seguridad	8%	3	4	0,24	0,32
Dependencia del proveedor	4%	3	3	0,12	0,12
Mantenimiento	7%	2	3	0,14	0,21
Cumplimiento normativo	3%	3	3	0,09	0,09
Trazabilidad	15%	4	4	0,60	0,60
Facilidad de evolución	6%	3	4	0,18	0,24
Adecuación al tamaño de la empresa	8%	3	5	0,24	0,40
Total sobre 5	**100%**			**3,29**	**3,84**

7.3 Justificación de pesos y notas

Criterio y peso	Por qué pesa eso	A · API propia + BD	B · Arquitectura híbrida
Funcionalidad (12%)	La solución tiene que cubrir pedido, cambio de estado, incidencia, agente de IA y revisión humana.	**Nota 4.** Podía cubrirlo todo, pero el agente y su revisión exigían programar más lógica a medida.	**Nota 5.** Los flujos de n8n, el agente y CoreAdmin están pensados para cubrir los tres escenarios y la revisión.
Coste estimado (8%)	Una empresa pequeña necesita un coste contenido, pero no tenemos precios comprobados.	**Nota 3.** Necesita alojamiento y base de datos, y más horas de desarrollo.	**Nota 3.** Requiere más servicios (n8n, Neon, Chartbrew y el backend), con menos horas de desarrollo. Sin cifras, no vemos motivo para dar ventaja a ninguna.

Criterio y peso	Por qué pesa eso	A · API propia + BD	B · Arquitectura híbrida
Esfuerzo de implantación (10%)	Una empresa pequeña dispone de poco tiempo y pocos recursos para implantar.	**Nota 2.** Hay que programar y probar la API, su base de datos y cada integración.	**Nota 3.** Los flujos se configuran de forma visual, aunque hicieron falta la outbox, la réplica de tickets, las rutas de back-office y pruebas.
Escalabilidad (4%)	El volumen esperado en una tienda pequeña es moderado.	**Nota 4.** Se puede ampliar a medida, con el esfuerzo técnico que suponga.	**Nota 3.** Es suficiente para este caso, pero está sujeta a los límites de los planes contratados.
Integración (15%)	Es el centro del caso: los datos deben circular entre sistemas sin intervención manual.	**Nota 4.** n8n llamaba a sus endpoints, y la API validaba, normalizaba y descartaba duplicados; cada transformación había que programarla.	**Nota 4.** Los pedidos y cambios de estado llegan a n8n por webhook, y este los valida y los escribe en Neon. No llega al 5 porque la outbox, la réplica de tickets y la reconciliación de pedidos siguen siendo código propio.
Seguridad (8%)	Se manejan datos de pedidos y secretos de integración.	**Nota 3.** Añade otra API expuesta cuya protección depende por completo del equipo.	**Nota 4.** Usa webhooks autenticados, cuyo secreto no llega al navegador, y un usuario de solo lectura (`dashboard_reader`) para Chartbrew. Las rutas de CoreAdmin exigen sesión de administrador.
Dependencia del proveedor (4%)	Importa, pero pesa menos en un prototipo de este tamaño. Una nota alta significa menos dependencia.	**Nota 3.** El código es propio, pero también dependía de n8n, Render y Neon.	**Nota 3.** Depende de n8n, Neon y Chartbrew. Se atenúa porque los flujos se exportan y los datos están en PostgreSQL.
Mantenimiento (7%)	Cada cambio debe poder aplicarse con pocos recursos.	**Nota 2.** El equipo tendría que mantener una API y una base de datos más.	**Nota 3.** Los flujos son visibles y fáciles de modificar, pero seguimos manteniendo el backend, la outbox, la réplica de tickets y las rutas.
Cumplimiento normativo (3%)	Afecta a los datos personales y a la supervisión humana. Pesa poco porque el prototipo usa datos de prueba y, al empatar, no favorece a ninguna alternativa.	**Nota 3.** Depende de cómo se implemente.	**Nota 3.** Depende de cómo se configure y de los proveedores usados. La revisión humana ayuda, pero no marca diferencia por sí sola.
Trazabilidad (15%)	La empresa debe poder comprobar qué ha pasado en cada ejecución y auditar las decisiones.	**Nota 4.** Registraba cada llamada de n8n (correcta, duplicada o fallida) en `registro_integracion`, además del historial de n8n.	**Nota 4.** Combina el historial de n8n, la outbox e `integration_logs`, aunque los rechazos de validación no quedan en `integration_logs`.
Facilidad de evolución (6%)	Deben poder añadirse eventos y métricas nuevas.	**Nota 3.** Es flexible, pero cada cambio requiere desarrollo y despliegue.	**Nota 4.** Se pueden añadir flujos y vistas SQL sin rehacer lo existente.
Adecuación al tamaño de la empresa (8%)	Tratamos UCAM Stereo como una tienda pequeña, sin un equipo técnico amplio.	**Nota 3.** Exige una dedicación técnica continua y difícil de asumir.	**Nota 5.** Equilibra control y simplicidad: conserva una base propia sin programar toda la integración.

7.4 Elección tecnológica

Recomendamos la alternativa B porque encaja mejor con lo que necesita UCAM Stereo: una tienda pequeña cuyos pedidos e incidencias deben llegar al back-office sin copiarlos a mano, dejando constancia de cada ejecución. Las dos alternativas resuelven igual de bien la integración y la trazabilidad, que suman el 30% del peso, porque ambas usan n8n y registran cada ejecución.

La diferencia está en lo que hay que construir y mantener. A exigía sostener otra API con su propia base de datos, que además duplicaba el back-office; B reutiliza el backend existente y deja la lógica de los flujos en n8n. Por eso B gana en adecuación al tamaño de la empresa, funcionalidad, esfuerzo de implantación y mantenimiento. B no elimina el desarrollo propio: seguimos manteniendo el backend, la outbox, la réplica de tickets y las rutas del back-office.

A sería preferible si la prioridad fuera tener el máximo control sobre el código y escalar a medida. Por eso supera a B en escalabilidad, aunque ese criterio pesa poco con el volumen y los recursos de esta empresa.

7.5 Solidez del resultado

Como los pesos son una decisión nuestra, hemos comprobado si el resultado cambia al modificarlos:

- **Pesos iguales.** B obtiene 3,67 y A 3,17.
- **Pesos de una empresa grande.** Funcionalidad y seguridad 15% cada una, dependencia del proveedor 12%, escalabilidad y cumplimiento normativo 10% cada uno, integración, trazabilidad y mantenimiento 8% cada uno, coste y evolución 5% cada uno, esfuerzo 4% y adecuación al tamaño 0%. B obtiene 3,66 y A 3,29.

En ambos casos gana B, así que la elección no depende de un reparto concreto de los pesos, aunque las notas siguen siendo una valoración nuestra.

7.6 Riesgos y límites de la valoración

- **Dependencia de servicios externos.** Dependemos de n8n, Neon, Render y Chartbrew, además de los límites de sus planes. Lo aceptamos porque los flujos se pueden exportar y los datos están en PostgreSQL, de modo que cambiar de herramienta no supondría empezar de cero.
- **Coste.** No hemos incorporado cifras porque no las hemos comprobado. Por eso ambas alternativas tienen la misma nota en este criterio.

8 Pruebas, gestión de errores y trazabilidad

8.1 Comprobaciones verificables en el código

En una revisión local asistida por IA del commit `92961a0` (04/11/2026) ejecutamos `npm test` en `backend`:

- **47 pruebas**: 46 superadas y 1 omitida, la de integración, que necesita PostgreSQL. La comprobación de precios compartidos entre frontend y backend también fue correcta.
- **Builds**: `npm run build` de la tienda y de CoreAdmin terminaron sin errores.
- **Prueba de integración contra PostgreSQL local**: falla al consultar los tickets con la cuenta de administración de la tienda, ruta que retiramos al trasladar la administración a CoreAdmin. Debemos actualizarla.

Ninguna prueba local sustituye las evidencias n8n–Neon–Telegram.

Mecanismo	Comportamiento implementado
Outbox transaccional	Pedido, cambio de estado o ticket y su evento se confirman juntos; si n8n falla, la compra no se revierte.
Idempotencia	`event_id` único en la outbox y en `orders`; cabecera `Idempotency-Key`; los flujos responden `already_processed`.
Reintentos	Espera de $5\,\text{s} \cdot 2^{n-1}$ (máximo 15 min); los registros bloqueados más de 5 min se recuperan.
Error terminal	Tras ocho intentos el evento pasa a `dead_letter` con `last_http_status` y `last_error`; se reactiva manualmente con la consulta de `N8N_PEDIDOS.md`.
Registro en back-office	`integration_logs` guarda `completed`, `pending_sync` y `failed` con `error_type`, mensaje y `execution_id`.
Rechazo de validación	La respuesta 400 no se escribe en `integration_logs`: queda en la outbox de la tienda (`last_error`) y se reintenta hasta `dead_letter`.

8.2 Evidencias de la defensa

No repetimos en esta memoria lo que mostramos en la presentación y la demostración. En ella enseñamos, con datos de prueba:

Elemento	Qué mostramos
Escenario 1	Pedido en la tienda, ejecución de n8n y filas en `orders`, `order_items` e `integration_logs`.
Escenario 2	Estado antes y después y su registro en `integration_logs`.
Escenario 3	Incidencia, aviso en Telegram y mensaje posterior a la aprobación.
Agente de IA	Propuesta con sus siete campos, aprobación o rechazo en CoreAdmin y ejecución.
Errores	Un evento `failed` o `pending_sync` y su reintento en la outbox.
Cuadro de mando	Panel de Chartbrew con los valores de las métricas.

9 Riesgos, limitaciones y mejoras futuras

Riesgo	Consecuencia	Medida actual / mejora
Caída de n8n o del plan gratuito	Eventos demorados; Render tarda en despertar.	Outbox con reintentos y `dead_letter`.
Doble vía de sincronización	Evidencias y métricas ambiguas; menor independencia.	Correlación por `event_id`; limitar la reconciliación a reparar sin reescribir.
Errores permanentes reintentados	Un 400 se reintenta ocho veces.	Pasar los 4xx directamente a `dead_letter` y registrarlos en `integration_logs`.
Exposición de credenciales	Acceso indebido a integraciones.	Secretos solo en Render y n8n; rotar el enlace de Chartbrew.
Salida errónea o manipulada de la IA	Propuesta inadecuada; texto del cliente con instrucciones.	Sin herramienta de envío; aprobación humana obligatoria.
Documentación desalineada	README, `render.yaml` y despliegue discrepan.	Revisar documentación y fijar commit final.

9.1 Limitaciones observadas

- En CoreAdmin, Pedidos e Incidencias trabajan con datos reales, pero Pedidos solo permite consultar, Integraciones muestra datos simulados y Logs no está implementado. El README afirma lo contrario y debemos corregirlo.
- El backend accede directamente a la base del back-office (reconciliación, incidencias y CoreAdmin): la separación de bases es real, pero no toda la comunicación pasa por n8n.
- Faltan en el repositorio los exports de los flujos de pedidos y del agente; Telegram notifica a un chat académico de prueba, no a cada cliente.
- `render.yaml` aún declara una PostgreSQL de Render mientras la guía de despliegue sitúa ambas bases en Neon. La prueba de integración con PostgreSQL está desactualizada.

9.2 Mejoras futuras

Proponemos las siguientes mejoras:

- Hacer de n8n la única vía de escritura de pedidos, dejando la reconciliación solo como auditoría.
- Registrar los rechazos de validación y mostrar `integration_logs` en la pantalla Logs.
- Sustituir los datos simulados de Integraciones por el estado de las dos outbox.
- Vincular el chat de Telegram de cada cliente mediante un enlace de alta.
- Migrar a planes sin caducidad antes de cualquier uso real.

9.3 Conclusiones provisionales

Con esta Tarea 2 hemos ampliado UCAM Stereo con una integración orientada a eventos y tolerante a fallos, un back-office con base propia, gestión de incidencias y un agente cuya propuesta solo se ejecuta tras una aprobación humana registrada. El código cubre los tres escenarios y lo hemos verificado con pruebas automáticas.

Anexo A. Uso de IA generativa y agéntica

A.1 Cómo hemos utilizado la IA

Este anexo actualiza el de la Tarea 1 y responde al apartado 10 del enunciado. Durante el proyecto hemos usado la inteligencia artificial de tres formas distintas, que conviene no confundir:

Apoyo al desarrollo	Componente del sistema	IA agéntica supervisada
Asistentes que nos ayudan a programar, depurar, documentar y revisar. No forman parte del producto. Apartados A.2 y A.3.	Un modelo que analiza las incidencias de la tienda dentro de un flujo de n8n. Apartado A.4.	Ese mismo análisis funciona como agente, con herramientas, límites y aprobación humana. Apartado A.4.

	Requisito del enunciado	Apartado		Requisito del enunciado	Apartado
a	Herramientas, modelo y versión	A.2, A.4.1	f	Acciones que puede realizar	A.4.3
b	Arquitectura agéntica y roles	A.4.1	g	Acciones prohibidas	A.4.3
c	Prompts o instrucciones	A.3, A.4.1	h	Mecanismo de revisión humana	A.4.4
d	Herramientas conectadas al agente	A.4.2	i	Errores detectados en las pruebas	A.5
e	Datos a los que accede el agente	A.4.2	j	Cambios para controlar esos errores	A.5

A.2 Herramientas utilizadas

Herramienta	Modelo y versión
Claude (Anthropic)	Opus 5.5 (esfuerzo alto) y Sonnet 5.5
ChatGPT / Codex (OpenAI)	GPT-5.6 Sol High

En todos los casos la IA ha sido un apoyo. Respondemos como grupo de las decisiones finales, de la implementación y de comprobar que todo funciona.

A.3 Ejemplos de utilización

Cada ejemplo indica qué pedimos, qué información recibió la herramienta y qué hicimos con el resultado.

- **Matriz de decisión (Claude).** Usamos Claude como apoyo para elaborar la matriz de decisión de implantación. Le proporcionamos la información disponible del proyecto y le pedimos un primer borrador que comparase las dos alternativas trabajadas durante el desarrollo (la API propia con base de datos independiente y la arquitectura híbrida) según los doce criterios del enunciado. Sobre ese borrador pedimos cambios y le solicitamos contrastar el documento con el repositorio y la memoria. Esa revisión detectó notas y descripciones que no coincidían con el código, por lo que las corregimos y recalculamos los resultados.
  `https://claude.ai/share/ffaee651-4c04-4c3f-95fb-e2fbbd64b114`
- **Arquitectura e inicio (ChatGPT).** Usamos ChatGPT como apoyo para organizar y definir la arquitectura del back-office. Consultamos los requisitos del enunciado para determinar cómo conectar la tienda con la aplicación administrativa. La IA ayudó a definir la separación de las bases de datos, el papel de n8n como herramienta de integración y el funcionamiento de CoreAdmin como back-office independiente. También elaboramos el documento ARQUITECTURA del repositorio.
  `https://chatgpt.com/share/6aebb021-c5f0-83ed-8218-5f33b0bee3ba`
- **README final (Claude Sonnet 5.5).** Modificamos el README final incluyendo los datos que ya estaban anteriormente (back-office y arquitectura).
  `https://claude.ai/share/6d257ecd-ddd6-4020-98aa-05049a6b1f95`
- **Agentes y flujos de n8n (ChatGPT).** Usamos ChatGPT para organizar y generar el agente de IA y los flujos de n8n de las automatizaciones del proyecto, como el mensaje de Telegram al aprobar una propuesta o el registro de las incidencias en el back-office. Un miembro del grupo supervisó y corrigió todo el trabajo para comprobar que funcionaba correctamente.
  `https://chatgpt.com/share/6aebb173-0194-83ed-b037-3eca60346c84`

Conversaciones heredadas de la Tarea 1: organización y GitHub, organización del frontend, implementación (1 y 2), scripts SQL (1 y 2) y memoria.

A.4 IA agéntica: el agente de incidencias

Describimos el agente a partir de la documentación, el código y el esquema de datos del repositorio.

A.4.1 Arquitectura y rol. Hay un agente que analiza una incidencia y propone qué hacer, sin ejecutarlo. Una persona decide después, y solo entonces n8n envía el aviso por Telegram.

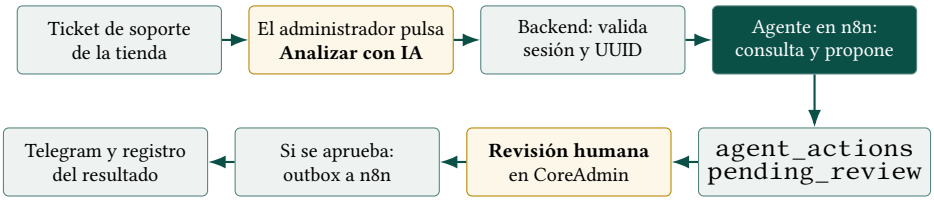


Figura 3. Recorrido de una incidencia. En verde oscuro, la IA; en crema, la intervención humana.

A.4.2 Herramientas y datos.

Elemento	Detalle
Herramienta de consulta	Lectura de la incidencia indicada.
Herramienta de acción	Registro de la propuesta en `agent_actions`.
Datos de entrada	El backend envía solo el UUID de la incidencia, con el secreto en una cabecera que nunca llega al navegador.
Datos accesibles	Incidencia en la base de datos del back-office, con datos de prueba.
Salida estructurada	Tipo de caso, datos consultados, acción propuesta, justificación, confianza, riesgos y estado.

A.4.3 Acciones permitidas y prohibidas. Reflejan el diseño documentado.

Puede	No puede
• Consultar la incidencia que se le indica. • Clasificar el caso y estimar su confianza. • Proponer una acción, justificarla y señalar riesgos. • Dejar la propuesta pendiente de revisión.	• Enviar mensajes sin aprobación. • Aprobar o ejecutar sus propias propuestas. • Modificar pedidos, pagos o importes. • Actuar sin que lo solicite un administrador.

A.4.4 Revisión humana. Un administrador autenticado ve la acción, la justificación, los riesgos y la confianza, y la aprueba o la rechaza. Quedan registrados `reviewed_by` y `reviewed_at`. La aprobación y el evento de salida se guardan en la misma transacción: si falla Telegram, no se pierde la decisión ni hay que volver a aprobar. Aprobar no significa ejecutar: `executed` solo se marca cuando la acción se ha efectuado (Figura 2, sección 5).

A.5 Errores detectados y cambios introducidos

En el apoyo al desarrollo.

Nº	Qué falló	Cómo lo detectamos	Qué cambiamos
E1	Dio por hecho que la API propia no usaba n8n y que sus registros estaban por desarrollar, y dio un 5 en trazabilidad a la opción elegida.	Al comparar la matriz con el repositorio, vimos que la API tenía rutas que llamaba n8n y sus propias tablas de registro.	Cambiamos cuatro notas y las descripciones de las alternativas.

En las pruebas del agente. Estas situaciones están previstas en el código, sus tests o la documentación; añadiremos los fallos de ejecuciones reales.

Situación controlada	Comportamiento
Identificador de incidencia no válido	Se rechaza antes de llamar a n8n.
Configuración incompleta	Devuelve un error que no revela secretos.
El agente no responde a tiempo	Se corta la espera y el error queda visible. El límite pasó de 30 a 60 segundos el 03/11/2026.
n8n no disponible	La compra y los tickets siguen funcionando, y se puede reintentar después.

A.6 Responsabilidad del grupo

Tratamos los resultados de la IA como propuestas que hay que revisar, y el apartado anterior muestra por qué. Cada integrante responde de lo que ha aportado y debe poder explicarlo en la defensa. Antes de enlazar una conversación, cada uno comprueba que no contiene contraseñas, tokens, cadenas de conexión ni datos personales reales.

Fuentes

[1] Enunciado oficial SIE, *Tarea 2: Integración, automatización de procesos e IA agéntica aplicada al back-office*, UCAM, 2026/27.
[2] Equipo UCAM Stereo, *Memoria UCAM Stereo, Tarea 1*, octubre de 2026; repositorio de la Tarea 1.